AWS SAA-C03 Study Guide

Module 4
Cost Optimization + Exam Preparation

How AWS billing works • Purchasing options • Cost-optimizing every layer • Data transfer rules • Cost tools • Cross-domain practice set • Hands-on labs • Exam-day strategy

1. How AWS Bills You (the model behind every cost question)

1.1 The three levers

Every AWS charge is some combination of **time** (hours an instance/volume/NAT exists, whether busy or idle), **capacity** (GB stored, IOPS provisioned, RCUs reserved), and **usage** (requests made, GB transferred, TB scanned). Cost optimization is about matching these to real need: stop paying for idle time (scale in, serverless), stop over-provisioning capacity (right-size, gp3, Intelligent-Tiering), and route usage through cheaper paths (endpoints, CloudFront, same-AZ).

CORE CONCEPT — The exam's cost reflex: When a question says MOST cost-effective, first list the answers that MEET the requirements, then pick the cheapest of THOSE. Never trade away a stated requirement (durability, latency, RTO) to save money — those distractors are deliberate. Managed/serverless usually wins for variable load; commitments win for steady load.

1.2 What is free that people forget

- Data transfer IN from the internet; traffic between resources in the same AZ over private IPs; S3/DynamoDB Gateway endpoints; ALB cross-zone balancing; CloudTrail management-event trail (one); AWS-managed KMS keys; CloudFront origin fetches from AWS origins; Trusted Advisor core checks; Cost Explorer.
- Charged even when idle: EC2/RDS running hours, EBS volumes (attached or not), unattached Elastic IPs and all public IPv4 addresses (hourly), NAT Gateway hours, provisioned IOPS, DynamoDB provisioned capacity, ELB hours, interface endpoints.

2. Compute Purchasing Options In Depth

2.1 The options and how to think about commitment

The trade is flexibility vs discount. On-Demand is the price of maximum flexibility. Commitments (RIs, Savings Plans) discount the portion of your usage you're sure about for 1 or 3 years. Spot sells spare capacity cheaply with the risk it's taken back. A mature account layers them: Savings Plans for the always-on baseline, On-Demand for the variable middle, Spot for the interruptible top.

Option	Discount	Commitment / flexibility	Best for
On-Demand	0%	None; per-second billing (Linux)	Unknown, short-lived, first deployments
Standard RI	up to ~72%	1 or 3 yr; locked to instance family + Region (size-flexible within family for Linux shared tenancy); resell on RI Marketplace	Steady, well-understood workloads you won't change
Convertible RI	up to ~66%	1 or 3 yr; CAN exchange family/OS/tenancy for equal-or-greater value	Steady but likely to evolve
Compute Savings Plan	up to ~66%	$/hour commitment; applies to ANY EC2 family/size/Region/OS, Fargate, AND Lambda	Most flexible commitment — default recommendation
EC2 Instance Savings Plan	up to ~72%	$/hour commitment to a family in a Region; size/OS/tenancy flexible	Steady usage of one family in one Region
Spot	up to ~90%	None — 2-minute reclaim notice; price set by pool supply	Fault-tolerant, stateless, interruptible: batch, CI/CD, rendering, big data, ASG mixed fleets
Dedicated Host	—	Physical server; visibility into sockets/cores; host affinity	BYOL per-socket/core licensing (Windows Server, SQL Server, Oracle); strict compliance
Dedicated Instance	—	Instances on hardware not shared with other accounts; no host control	Compliance requiring physical isolation without license needs

Option	Discount	Commitment / flexibility	Best for
Capacity Reservation	0% (On-Demand rate, stackable with SPs/RIs)	Reserve capacity in an AZ for any duration	Guarantee capacity for events/DR; not a discount tool

- Payment options for RIs/SPs: All Upfront (biggest discount) > Partial Upfront > No Upfront. RIs and SPs are shared across accounts in an Organization with consolidated billing (unless sharing is turned off).
- Spot best practices: diversify across many instance types and AZs, use capacity-optimized allocation, checkpoint work, handle the 2-minute interruption notice (metadata / EventBridge), and never put a single stateful component on Spot. Spot Fleet/ASG can mix On-Demand base + Spot.

EXAM TIP: 'Steady 24/7 for 3 years, max savings' → Standard RI or EC2 Instance SP (all upfront). 'Also runs Fargate/Lambda' or 'may change families/Regions' → Compute SP. 'Interruptible / batch / stateless' → Spot. 'Software licensed per socket' → Dedicated Host. 'Guarantee capacity for a launch event' → Capacity Reservation.

3. Cost-Optimizing Each Layer

3.1 Compute

- **Right-size** with Compute Optimizer (ML recommendations for EC2, ASG, EBS, Lambda memory) — the most common answer to 'instances at 10% CPU'. **Graviton** for ~20–40% better price/performance. **Auto Scaling** so you never pay for peak 24/7; scheduled scaling/Instance Scheduler to stop dev/test nights and weekends. **Serverless** (Lambda, Fargate, Aurora Serverless) for sporadic or spiky load — no idle cost. **Lambda tuning:** more memory can finish faster and cost less; Power Tuning finds the sweet spot.

3.2 Storage

- **S3:** lifecycle policies + Intelligent-Tiering (Module 3a); delete incomplete multipart uploads; expire old versions; use Storage Class Analysis and S3 Storage Lens to find waste; requester-pays buckets shift transfer cost to consumers.
- **EBS:** gp3 over gp2/io1 unless IOPS demands it; delete unattached volumes and orphaned snapshots (Data Lifecycle Manager retention); st1/sc1 for sequential cold data; snapshots archive tier for rarely-restored snapshots.
- **EFS:** Infrequent Access + Archive tiers with lifecycle; One Zone for non-critical; Elastic throughput instead of over-provisioned. **FSx:** scratch Lustre for temporary jobs; right-size throughput capacity.

3.3 Database

- RDS/Aurora RIs for steady load; Aurora Serverless v2 for variable; stop dev RDS instances (they auto-restart after 7 days — snapshot + delete for long pauses); Aurora I/O-Optimized when I/O charges dominate; storage auto-scaling to avoid over-allocating.
- DynamoDB: provisioned + auto scaling for predictable traffic (cheaper), on-demand for spiky/unknown; reserved capacity for steady provisioned tables; TTL to purge junk; Standard-IA table class for rarely accessed tables; eventually consistent reads cost half; DAX/ElastiCache cut read units.
- Offload analytics: replica for reporting; export/ETL to S3 and query with Athena instead of running a warehouse 24/7; Redshift Serverless / pause clusters for intermittent use.

3.4 Network & data transfer — the most-tested cost topic

Traffic path	Cost	Optimization
Internet → AWS (ingress)	Free	—

Traffic path	Cost	Optimization
AWS → internet (egress)	Per-GB, tiered; the big one	CloudFront in front (cheaper egress + caching); compress; cache at edge
Same AZ, private IP	Free	Keep chatty tiers co-located (when HA design allows)
Cross-AZ within Region	Per-GB each direction	Reduce cross-AZ chatter; AZ-aware routing; accept for HA
Cross-Region	Per-GB	Only replicate what DR/compliance needs
Via NAT Gateway	Hourly + per-GB processed	Gateway endpoints for S3/DynamoDB (free), interface endpoints for other heavy AWS API traffic, or route through one NAT per AZ (avoid cross-AZ NAT hops)
Via Direct Connect	Lower per-GB than internet egress	Route bulk hybrid egress over DX
VPC peering / TGW	Peering: standard cross-AZ/Region data rates; TGW adds per-attachment hour + per-GB	Peering is cheaper for few VPCs; TGW for manageability at scale

EXAM TIP: 'High NAT Gateway charges' → check what's flowing: S3/DynamoDB traffic → Gateway endpoint; other AWS services → interface endpoints; instances in AZ-A using NAT in AZ-B → add a NAT per AZ. 'High data transfer out to users' → CloudFront.

4. Cost Visibility & Governance Tools

Tool	What it answers	Exam trigger
Cost Explorer	Where did the money go? Visualize by service/account/tag; 12-month forecast; RI/SP recommendations and utilization/coverage reports	'Analyze spending trends', 'which team/project costs what'
AWS Budgets	Am I about to overspend? Cost/usage/RI-SP budgets with alerts at thresholds (actual or forecast); Budget Actions apply IAM/SCP restrictions or stop instances automatically	'Alert when forecast exceeds', 'stop resources when budget hit'
Cost Anomaly Detection	Is something unusual happening? ML flags spend spikes to SNS/email	'Notify on unexpected cost increases'
Cost & Usage Report (CUR)	The most granular line-item data, delivered to S3; query with Athena, visualize in QuickSight	'Detailed hourly/resource-level billing data', 'chargeback with custom SQL'
Cost allocation tags	Attribute costs to owners — tag resources, ACTIVATE the tags in Billing, then filter/group everywhere	'Track costs per department/project'
Compute Optimizer	What's over/under-provisioned? Right-sizing for EC2, ASG, EBS, Lambda, ECS Fargate	'Identify underutilized instances'
Trusted Advisor	Best-practice checks incl. idle LBs, low-utilization instances, unattached volumes, RI optimization (full checks need Business/Enterprise support)	'Automated best-practice recommendations'
Pricing Calculator	Estimate before building	'Estimate monthly cost of a proposed architecture'
Consolidated billing / Organizations	One bill; volume-tier aggregation; RI/SP sharing across accounts	'Reduce cost across many accounts'
Savings Plans / RI utilization reports	Are my commitments being used?	'Verify commitment coverage'

Support plans (appear in cost questions)

Basic (free) → **Developer** (business-hours email) → **Business** (24/7 phone/chat, full Trusted Advisor, 1-hour production-down response) → **Enterprise On-Ramp** → **Enterprise** (Technical Account Manager, 15-min business-critical response, concierge). 'Needs a TAM / 15-minute response' → Enterprise; 'cheapest plan with full Trusted Advisor and 24/7 support' → Business.

5. Reference Architecture: Same App, Cost-Optimized

Take the 3-tier web app from Module 1 and the order-processing system from Module 2, then apply the cost lens without weakening any requirement:

- **Edge:** CloudFront caches static assets and API GETs → cuts origin egress (the largest variable bill) and lets the web tier shrink.
- **Web/app tier:** ASG with mixed instances: 2 On-Demand Graviton base covered by a Compute Savings Plan, scale-out on Spot across 4 instance types. Target-tracking on requests/target; scheduled scale-in overnight for internal tools.
- **Workers:** SQS-driven Fargate Spot tasks; no idle cost between bursts. Long jobs on AWS Batch with Spot.
- **Data:** Aurora Serverless v2 for the variable catalog, RDS RI for the steady core ledger, reporting moved to a read replica then to S3 + Athena (Parquet) for history. DynamoDB sessions on on-demand with TTL.
- **Storage:** uploads to Intelligent-Tiering; derived images One Zone-IA; logs lifecycle to Glacier; gp3 everywhere; Data Lifecycle Manager prunes snapshots.
- **Network:** Gateway endpoints for S3/DynamoDB; one NAT per AZ; interface endpoints for high-volume SQS/CloudWatch traffic instead of NAT.
- **Governance:** cost allocation tags by team, Budgets with forecast alerts, Anomaly Detection to SNS, monthly Compute Optimizer review.

Notice nothing lost HA (still multi-AZ), durability, or security — that's the constraint every cost question silently enforces.

6. Cross-Domain Practice Set

These mix all four domains the way the real exam does. Extract requirements + qualifier, eliminate, then choose.

Practice Q1. A company runs 40 m5.xlarge instances 24/7 for a steady workload it expects to keep for 3 years, plus a Fargate-based API with unpredictable load. It wants maximum savings with minimum management. Which purchasing strategy?

✗ Standard RIs for everything — RIs don't cover Fargate; locks 40 instances to m5 for 3 years.

✗ Spot for the steady fleet — interruption risk for a production workload with no stated tolerance.

✓ 3-year Compute Savings Plan sized to the steady baseline — discounts the m5s AND Fargate, survives family changes, one commitment to manage. 'EC2 + Fargate/Lambda in one commitment' is the Compute SP fingerprint.

Practice Q2. A monthly financial report job reads 2 TB from an S3 data lake and takes 6 hours on a Redshift cluster that otherwise sits idle. Reduce cost with minimal change.

✗ Bigger Redshift cluster — faster but costlier and still idle 29 days.

✗ Move data to RDS — wrong engine for 2 TB analytical scans.

✓ Redshift Serverless (or pause/resume the provisioned cluster) so compute is billed only during the job; alternatively Athena over Parquet-partitioned data if queries are simple. Idle time is the waste — eliminate it.

Practice Q3. Instances in private subnets push 50 TB/month of logs to S3 through a NAT Gateway. The NAT bill is the largest line item. Fix with no security regression?

✗ Give instances public IPs — security regression, and internet egress still costs.

✗ Interface endpoint for S3 — works but charges per-GB; there's a free option.

✓ S3 Gateway VPC endpoint in the private route tables — free, private, traffic bypasses the NAT entirely. Any 'NAT costs + S3/DynamoDB' question ends here.

Practice Q4. A media company must store 500 TB of raw footage. Editors access about 5% in the first month; afterward access is rare but must be immediate when requested. Compliance requires 3 AZs. MOST cost-effective?

✗ S3 Standard — pays hot prices for 95% cold data.

✗ One Zone-IA — fails the 3-AZ requirement.

✗ Glacier Flexible — minutes-to-hours retrieval breaks 'immediate'.

✓ S3 Intelligent-Tiering (or Standard → Standard-IA lifecycle at 30 days). Intelligent-Tiering fits best because access is partly unpredictable; both keep millisecond retrieval across 3 AZs. Requirements first, then cheapest.

Practice Q5. A startup's Lambda functions connect to an RDS PostgreSQL database. Under load, the database rejects connections ('too many connections'). Fix?

✗ Scale RDS vertically — raises the connection ceiling temporarily and costs continuously.

✗ Increase Lambda concurrency — creates more connections; worsens it.

✓ Put RDS Proxy between Lambda and RDS — pools and multiplexes connections, and IAM-authenticates. Lambda + relational DB connection exhaustion = RDS Proxy.

Practice Q6. A regulated firm needs to prove to auditors which user made specific API calls in any of its 25 accounts over the last year, with tamper-evident logs. What should be in place?

✗ CloudWatch Logs from applications — app logs, not AWS API audit.

✗ AWS Config — tracks resource configuration state, not who called what.

✓ An Organization trail in CloudTrail delivering to a central S3 bucket with log file integrity validation enabled (and Object Lock/restricted bucket policy). 'Who called which API' = CloudTrail; 'across all accounts' = org trail.

Practice Q7. A web app on EC2 across 2 AZs uses an ALB. A new requirement: users must sign in with the company's existing SAML identity provider before reaching the app, with minimal code changes. Solution?

✗ Cognito Identity Pools — issues AWS credentials, doesn't gate HTTP requests.

✗ Rewrite the app to validate SAML — maximal code change.

✓ ALB authentication action with a Cognito User Pool federated to the SAML IdP (or OIDC directly) — the ALB redirects unauthenticated users to sign in and forwards claims in headers; the app barely changes. ALB can authenticate; use it.

Practice Q8. A company must migrate a 4 TB on-prem Oracle database to Aurora PostgreSQL with under 30 minutes of downtime. Which combination?

✗ Export/import dump files — hours of downtime for 4 TB.

✗ Snowball — offline transfer can't provide near-zero downtime cutover.

✓ Schema Conversion Tool to convert the Oracle schema, then DMS full load + ongoing CDC replication; cut over once the replica is caught up. Heterogeneous + minimal downtime = SCT + DMS with CDC.

Practice Q9. An IoT fleet sends 100,000 small messages per second. Data must be queryable with SQL within 2 minutes of arrival and retained for 5 years at lowest cost. Design?

✗ Write directly to RDS — write rate and 5-year growth are hostile to OLTP.

✗ Kinesis Data Streams to EC2 consumers writing files — unnecessary servers.

✓ Data Firehose → S3 with Parquet conversion and dynamic partitioning (60-second buffers meet '2 minutes'), Glue catalog, Athena for SQL, lifecycle to Glacier for long-term retention. Near-real-time + cheap retention + SQL = Firehose + S3 + Athena.

Practice Q10. A company's DR requirement for its e-commerce platform is RTO 10 minutes and RPO 1 minute, in a second Region, at the lowest cost that meets the targets.

✗ Backup & Restore — hours of RTO.

✗ Multi-site active/active — meets targets but doubles cost; not lowest.

✓ Warm standby: scaled-down full stack in the DR Region behind Route 53 failover, Aurora Global Database (RPO ~1 s), scale up on failover within minutes. Pilot light risks exceeding 10 minutes since app tier must launch from scratch; warm standby is the cheapest band that reliably fits.

Practice Q11. Developers need to deploy container images to a production cluster. Security requires that images are scanned for vulnerabilities and that production tasks can only read one specific S3 bucket. Which combination?

✗ Scan with Macie — Macie finds PII in S3, not image CVEs.

✗ Attach S3 permissions to the ECS instance role — every task on the host inherits them.

✓ Store images in ECR with scan-on-push (Inspector-powered), and grant S3 access via an ECS TASK role scoped to that bucket. Image CVEs = ECR/Inspector; per-task least privilege = task role.

Practice Q12. A SaaS provider wants to expose its API, hosted behind an NLB in its VPC, to hundreds of customers' VPCs privately, without peering or overlapping-CIDR headaches. Solution?

✗ VPC peering with each customer — hundreds of non-transitive peerings and CIDR conflicts.

✗ Public API with IP allow-listing — traverses the internet; 'privately' fails.

✓ AWS PrivateLink: create an endpoint service on the NLB; customers create interface endpoints in their VPCs. Scales to thousands, ignores CIDR overlap, traffic never leaves AWS. 'Expose my service privately to many VPCs' = PrivateLink.

7. Hands-On Labs Checklist (free tier friendly)

Reading builds recognition; building builds understanding. Budget 10–15 hours across these; each cements a cluster of exam facts. Delete resources afterward and set a $10 AWS Budget alert before you start.

#	Lab	What it cements	~Time
1	Account hygiene: enable MFA on root, create an admin IAM user via Identity Center, set a Budget alert, enable CloudTrail org/account trail	Root vs IAM, Identity Center, Budgets, CloudTrail	45 min
2	Build a VPC by hand (not the wizard): /16 VPC, 2 public + 2 private subnets in 2 AZs, IGW, route tables, NAT Gateway in one public subnet, S3 Gateway endpoint. Launch an instance in each subnet; test what can reach what	Everything in Module 1 §3 — the highest-value lab	2 h
3	IAM role → EC2 → S3: create a bucket, a role with least-privilege policy, attach to an instance, use the CLI from the instance; then break the policy (remove ListBucket) and observe	Roles, policy anatomy, bucket vs object ARNs	45 min
4	SG chaining: ALB SG → app SG → (RDS free tier) DB SG referencing each other; add a NACL deny for your own IP and watch it block	SG vs NACL, stateful vs stateless	1 h
5	ALB + Auto Scaling: launch template with user data installing a web page showing the AZ; ASG across 2 AZs with target tracking; stress with a load tool; terminate an instance and watch replacement	Module 2 §2–3	1.5 h
6	SQS + Lambda: create a queue with a DLQ, a Lambda consumer via event source mapping, publish messages; make the function throw errors to see retries and DLQ; add an SNS topic fanning out to two queues	Visibility timeout, DLQ, fan-out	1 h
7	S3 features: enable versioning, upload/overwrite/delete and recover; lifecycle rule to IA; Block Public Access on; CloudFront distribution with OAC to the private bucket; generate a presigned URL	Module 3a §2	1.5 h

#	Lab	What it cements	~Time
8	RDS Multi-AZ (free tier db.t3.micro): create, force a reboot-with-failover, observe endpoint stays the same; add a read replica	Multi-AZ vs replica	1 h
9	DynamoDB + Lambda + API Gateway: table with partition/sort key, HTTP API → Lambda writing/reading items, enable TTL and Streams	Serverless API pattern	1.5 h
10	Athena on logs: enable ALB access logs to S3 (or use the sample CloudTrail logs), Glue crawler, run SQL in Athena; note bytes scanned	Data lake basics, Athena cost model	1 h
11	Cost tour: open Cost Explorer, activate a cost allocation tag, look at Trusted Advisor and Compute Optimizer	Module 4 §4	30 min

8. Putting It Together: Study Plan & Exam-Day Strategy

8.1 A 4-week plan using this guide

- **Week 1:** Modules 1 and 2 with labs 1–6. After each module, write the self-check answers from memory before checking.
- **Week 2:** Modules 3a and 3b with labs 7–10. Start one practice-exam bank (Tutorials Dojo, or the free AWS Skill Builder official practice set).
- **Week 3:** Module 4 + lab 11. Take 2 full timed practice exams. For EVERY question (right or wrong), write one line: which requirement/qualifier decided it. Re-read the cheat-sheet sections tied to your weak domain.
- **Week 4:** 2–3 more timed exams targeting 80%+ consistently; revisit the keyword map in the cheat sheet daily; last two days, light review only.
- Signal you're ready: scoring 80–85% on practice exams you haven't seen before, and being able to explain why each distractor is wrong.

8.2 Reading a question in 90 seconds

- 1. Read the final sentence first — it holds the actual ask and the qualifier (cost-effective / least overhead / most secure / highly available).
- 2. Scan the scenario for hard constraints: numbers (RPO/RTO, TB, latency, users), named technologies (Windows, Kubernetes, Oracle, Kafka), and words like 'private', 'no internet', 'cannot change the application'.
- 3. Eliminate answers that violate any constraint, regardless of how attractive they sound.
- 4. Among survivors, apply the qualifier. Ties break toward managed/serverless for overhead questions and toward the cheaper valid option for cost questions.
- 5. Flag if unsure and move on; return with leftover time. Never leave blanks — there's no penalty for guessing.

8.3 Distractor patterns to recognize

- Right service, wrong scope: security group with a 'deny' rule; CNAME at the zone apex; Gateway endpoint for a non-S3/DynamoDB service; Lambda for a 30-minute job.
- Right idea, more work than needed: 'write a Lambda to rotate secrets' (Secrets Manager does it), 'build a script to copy snapshots' (AWS Backup), 'set up EC2 with Kafka' (MSK/Kinesis).
- Solves the wrong pillar: Multi-AZ offered for a read-performance problem; a bigger instance offered for an availability problem; Spot offered for a stateful database.
- Violates a stated requirement to save money: One Zone-IA when 'multi-AZ' is required; Glacier when 'immediate' access is required; VPN when 'consistent bandwidth' is required.

8.4 Logistics

- Book at a Pearson VUE center if you can — fewer online-proctoring hiccups. Arrive/log in 30 minutes early with ID. Request the +30 minute ESL accommodation before scheduling if English isn't your first language.
- 130 minutes for 65 questions: aim to finish a first pass by minute 100, leaving 30 for flagged questions.
- Results typically arrive within a few hours to 5 business days. Certification is valid 3 years.

You now have the complete set: the cheat sheet for compression and recall, and Modules 1–4 for understanding. Alternate between them — read a module section, then drill the matching cheat-sheet table from memory. That loop, plus the labs and timed practice, is what passes this exam.